

Typinformation

Man kan även få information från en datatyp om vilken typ det är frågan om. I normala fall vet man vilken datatyp det är då det är frågan om primitiva datatyper, men vid arvshierarkier och med pekare till basklasser kan man inte alltid vara säker. Man kan i sådana fall använda sig av t.ex. någon inbyggd metod eller så `dynamic_cast` för att försöka konvertera objektet till någon viss klass. Ett alternativt sätt är att använda en annan ny finess hos C++, man kan nämligen få ut en *id* för varje objekt som säger vilken datatyp det är frågan om. Denna id kan användas för att jämföra om två objekt är av samma typ. Funktionen som ger informationen heter `typeid()` och returnerar en referens till en klass `const type_info`. Denna funktion kan användas efter att headerfilen `<typeinfo>` inkluderats. Ett exempel på hur `typeid()` kan användas:

```
#include <typeinfo>

void foobar (A * Data) {
    // hämta information om 'Data'
    const type_info & Info = typeid ( *Data );
    cout << "Typen = " << Info.name () << endl;
}
```

Vi måste ha en referens till en `type_info` för returvärdet från `typeid()`. Notera att funktionen returnerar typen på dess parameter, så skickar vi t.ex. en pekare får vi ut som information att det är en pekare till någon datatyp, därför avrefereras `Data` innan den skickas som parameter. Klassen `type_info` har överlagrade operatorer (se [Kapitel 21](#)) som tillåter jämförelse (`==` och `!=`) mellan olika instanser av `type_info`. En metod `name()` finns tillgänglig ifall man vill använda en strängrepresentation av datatypen till något, t.ex. till en `map` över olika klasser.